

Nesne Tabanlı Programlama

OOP Kavramları, Naming Convention, Class & Object, Methods ve Constructors

Java ile Nesne Tabanlı Programlama · Dr. Öğr. Üyesi Baki Gökgöz · Gümüşhane Üniversitesi

Bu Derste Neler Öğreneceğiz?

OOP Kavramları

Object, class, inheritance, polymorphism, abstraction, encapsulation ve nesneler arası ilişkiler.

Naming Convention

Class, method, variable, constant ve package isimlendirme standartları.

Class ve Object

Class tanımlama, object oluşturma, reference variable ve object initialization.

Methods

Method anatomisi, access modifier'lar, static/instance method, parametreler ve return.

Constructors

Default, no-arg, parameterized ve copy constructor; constructor overloading.

Alıştırmalar

Çıktı tahmini, sık yapılan hatalar ve özet.

Bölüm 1



OOP Kavramları

Gerçek dünyayı nesnelerle modellemek

Procedural vs Object-Oriented Programlama

Kriter	Procedure-Oriented (ör. C)	Object-Oriented (ör. Java)
Temel birim	Fonksiyon (function)	Object ve class
Veri ve davranış	Ayrı: veri fonksiyonlara parametre olarak gider	Birlikte: veri ve method'lar aynı class'ta
Veri erişimi	Global veriye her yerden erişilebilir	Data hiding: private alanlar korunur
Büyük projelerde bakım	Kod büyüdükçe yönetimi zorlaşır	Modüler yapı; geliştirme ve bakım kolay
Yeniden kullanım	Kopyala-yapıştır eğilimi	Inheritance ve composition ile yeniden kullanım
Gerçek dünya modellemesi	Dolaylı	Doğrudan: Öğrenci, Hesap, Araba…

Tarihçe: Simula (1967) ilk object-oriented dil kabul edilir; Smalltalk her şeyin object olduğu ilk "saf" OOP dilidir. Bugün Java, C#, C++, Python ve PHP yaygın OOP dilleridir.

OOP'nin Temel Kavramları

Object

State (durum) ve behavior (davranış) sahibi varlık: kalem, araba, banka hesabı.

Class

Object'lerin şablonu (blueprint). Hangi field ve method'ların olacağını tanımlar.

Inheritance

Bir class'ın başka bir class'ın özelliklerini devralması (IS-A). Kod tekrarını azaltır.

Polymorphism

Aynı işin farklı biçimlerde yapılması: overloading (compile-time) ve overriding (runtime).

Abstraction

İç detayları gizleyip sadece işlevi göstermek: abstract class ve interface.

Encapsulation

Veri ve kodu tek birimde toplamak, veriyi private yapıp getter/setter ile korumak.

Object: State, Behavior ve Identity

- **State (durum)**: object'in verileri → field'lar (`name`, `color`, `age`).
- **Behavior (davranış)**: object'in yapabildikleri → method'lar (`bark()`, `eat()`).
- **Identity (kimlik)**: her object'in JVM içindeki benzersiz kimliği; aynı değerlere sahip iki object yine de **farklı** object'lerdir.
- Object fiziksel (araba, kalem) veya mantıksal (banka hesabı, sipariş) olabilir.
- Object, bir class'ın **instance**'ıdır ve bellekte (heap) yer kaplar.

`myDog : Dog`

State (field'lar)

```
name   = "Rocky"
breed  = "Golden"
age    = 3
```

Behavior (method'lar)

```
bark()
eat()
wagTail()
```

Identity: JVM'deki benzersiz kimlik

İlk Örnek: Class Tanımlama ve Object Oluşturma

```java
class Dog {
    String name;  // state (field)

    void bark() {  // behavior (method)
        System.out.println(name + " says Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();  // object oluştur
        myDog.name = "Rocky";  // field'a değer ata
        myDog.bark();  // method çağır

        Dog other = new Dog();
        other.name = "Karabaş";
        other.bark();
    }
}
```

Çıktı (Output)

```
Rocky says Woof!
Karabaş says Woof!
```

- **`Dog`** bir **class**'tır; **`myDog`** ve **`other`** bu class'tan üretilen iki ayrı **object**'tir.
- **`new Dog()`** heap'te yeni bir object oluşturur.
- Nokta operatörü (**`.`**) ile object'in field ve method'larına erişilir.

Class Tanımının Parçaları



- **Modifier**: class `public` veya default (package-private) erişimli olabilir.
- **class keyword** ve **class adı**: konvansiyon gereği büyük harfle başlar (PascalCase).
- **extends**: yalnızca **bir** parent class'tan türetilebilir (Java'da çoklu class inheritance yok).
- **implements**: virgülle ayrılmış **birden fazla** interface uygulanabilir.
- **Body `{ }`**: field'lar, method'lar, constructor'lar, block'lar ve nested class'lar.

Inheritance (Kalıtım): IS-A İlişkisi

```java
class Vehicle {                        // superclass (parent)
    void start() { System.out.println("Vehicle is starting..."); }
    void stop()  { System.out.println("Vehicle is stopping..."); }
}

class Car extends Vehicle {            // subclass (child)
    void honk()  { System.out.println("Car is honking!"); }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car();
        myCar.start();      // Vehicle'dan miras
        myCar.stop();       // Vehicle'dan miras
        myCar.honk();       // Car'ın kendi method'u
    }
}
```

Çıktı (Output)

```
Vehicle is starting...
Vehicle is stopping...
Car is honking!
```

- "Car **bir** Vehicle'dır" → **IS-A** ilişkisi.
- **`Car`**, **`start()`** ve **`stop()`** method'larını yeniden yazmadan kullanır → **code reusability**.
- Inheritance, runtime polymorphism'in temelidir.

Polymorphism (Çok Biçimlilik)

```java
class Animal {
    void sound() { System.out.println("..."); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Hav hav"); }
}
class Cat extends Animal {
    @Override
    void sound() { System.out.println("Miyav"); }
}

public class Main {
    public static void main(String[] args) {
        Animal[] animals = { new Dog(), new Cat(), new Animal() };
        for (Animal a : animals) {
            a.sound();  // hangi sound()? runtime'da karar verilir
        }
    }
}
```

Çıktı (Output)

```
Hav hav
Miyav
...
```

- Aynı `a.sound()` çağrısı, object'in **gerçek tipine** göre farklı davranır.
- **Runtime polymorphism**: method overriding ile.
- **Compile-time polymorphism**: method overloading ile (aynı isim, farklı parametre).

Abstraction (Soyutlama)

```java
abstract class Animal {
    abstract void makeSound();          // gövdesiz: "ne" yapılacak
    void breathe() {                    // concrete method
        System.out.println("Animal is breathing...");
    }
}

class Dog extends Animal {
    @Override
    void makeSound() {                  // "nasıl" yapılacak
        System.out.println("Dog barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.breathe();
        myDog.makeSound();
        // Animal a = new Animal();  -> HATA: abstract
    }
}
```

Çıktı (Output)

```
Animal is breathing...
Dog barking
```

- Kullanıcı **ne** yapıldığını bilir, **nasıl** yapıldığını bilmesi gerekmez (telefonla arama örneği).
- `abstract` class'tan doğrudan object oluşturulamaz.
- Java'da abstraction: **abstract class** ve **interface**.

Encapsulation (Kapsülleme)

```java
class BankAccount {
    private double balance;            // dışarıdan erişilemez

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        } else {
            System.out.println("Geçersiz miktar!");
        }
    }
    public double getBalance() { return balance; }   // getter
}

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount();
        acc.deposit(500);
        acc.deposit(-100);
        // acc.balance = 1000000;  -> HATA: private
        System.out.println("Bakiye: " + acc.getBalance());
    }
}
```

Çıktı (Output)

```
Geçersiz miktar!
Bakiye: 500.0
```

- Veri **`private`** yapılır, erişim **kontrollü** public method'larla sağlanır.
- Setter/deposit içinde **doğrulama** yapılarak geçersiz durum engellenir.
- Tüm field'ları private olan class'lara **JavaBean** denir.

Tasarım Kavramları: Coupling ve Cohesion

Coupling (Bağımlılık)

- Bir class'ın başka bir class hakkında ne kadar bilgi sahibi olduğu / ona ne kadar bağımlı olduğu
- **Tight coupling**: class doğrudan diğer class'ı oluşturur ve onun detaylarına bağlıdır
- **Loose coupling** hedeflenir: interface'ler ve access modifier'lar bağımlılığı azaltır
- Hedef: **düşük coupling**

Cohesion (Bütünlük)

- Bir class/method'un **tek ve iyi tanımlı** bir işe ne kadar odaklandığı
- **High cohesion**: `Calculator` yalnızca hesap yapar
- `java.io` paketi yüksek cohesion'a örnektir (hepsi I/O ile ilgili)
- Hedef: **yüksek cohesion**

Nesneler Arası İlişkiler

İlişki	Anlam	Örnek	Bağın gücü
Inheritance (IS-A)	Bir class diğerinin türüdür	Car IS-A Vehicle	Class düzeyinde
Association	Object'ler birbirini kullanır/bilir	Ülke – Başbakan (one-to-one)	Genel ilişki
Aggregation (HAS-A)	Parça bağımsız da yaşayabilir	Üniversite – Bölüm	Zayıf
Composition (HAS-A)	Parça bütün olmadan anlamsızdır	Araba – Motor	Güçlü

Association türleri: one-to-one, one-to-many, many-to-one, many-to-many. Association tek yönlü (unidirectional) veya çift yönlü (bidirectional) olabilir. Aggregation ve composition, association'ın özel hâlleridir.

Aggregation ve Composition Kodda

Aggregation: parça dışarıdan gelir

```
class Department {
    String name;
    Department(String n) { name = n; }
}
class University {
    Department dept;  // HAS-A
    University(Department d) {  // dışarıdan
        dept = d;
    }
}
```

- **`Department`** önce oluşturulur, **`University`**'ye verilir.
- University yok olsa da Department başka yerde kullanılabilir.

Composition: parça içeride oluşur

```
class Engine {
    void start() {
        System.out.println("Motor çalıştı");
    }
}
class Car {
    private final Engine engine
            = new Engine();   // içeride
    void drive() { engine.start(); }
}
```

- Motor, araba ile birlikte oluşur ve onunla yaşar.
- Java'da object'ler "silinmez"; erişilemez hâle gelince **garbage collector** temizler.

Bölüm 2



Naming Convention

Okunabilir kodun ilk adımı: doğru isimlendirme

Java Naming Convention Kuralları

Identifier	Kural	Örnek
Class	Büyük harfle başlar, isim (noun) olur; PascalCase	`Employee, BankAccount, String`
Interface	Büyük harfle başlar, genelde sıfat (adjective)	`Runnable, Comparable, Printable`
Method	Küçük harfle başlar, fiil (verb) olur; camelCase	`print(), calculateArea(), getName()`
Variable	Küçük harfle başlar; camelCase; _ veya $ ile başlamaz	`firstName, totalPrice, id`
Constant	Tamamı büyük harf; kelimeler _ ile ayrılır	`MAX_SIZE, MIN_AGE, PI`
Package	Tamamı küçük harf; ters domain adı	`java.util, tr.edu.gumushane.proje`

Bunlar convention'dır (gelenek), kural değildir: compiler zorlamaz. Ancak endüstri standardıdır; IDE'ler, statik analiz araçları ve ekip arkadaşlarınız bu isimlendirmeyi bekler.

System.out.println() İsimlendirmesi Bize Ne Söyler?



PascalCase

Her kelime büyük harfle: `StudentRecord`

camelCase

İlk kelime küçük: `studentName`, `printInfo()`

UPPER_SNAKE_CASE

Sabitler: `MAX_PRIORITY`

lowercase

Paketler: `com.example.app`

Kötü ve İyi İsimlendirme

✗ Convention'a uymayan

```
class student_record {
    int X;
    static final int maxsize = 50;
    void Print_Info() {
        System.out.println(X);
    }
}
```

- Derlenir ama okuyan kişiyi yanıltır: **`X`** ne?
- **`maxsize`** sabit mi, variable mı?

✓ Convention'a uygun

```
class StudentRecord {
    int studentId;
    static final int MAX_SIZE = 50;
    void printInfo() {
        System.out.println(studentId);
    }
}
```

- Adından ne olduğu anlaşılıyor.
- Tek harfli isimler (**`x`**, **`y`**) yalnızca loop sayacı gibi kısa ömürlü yerlerde.

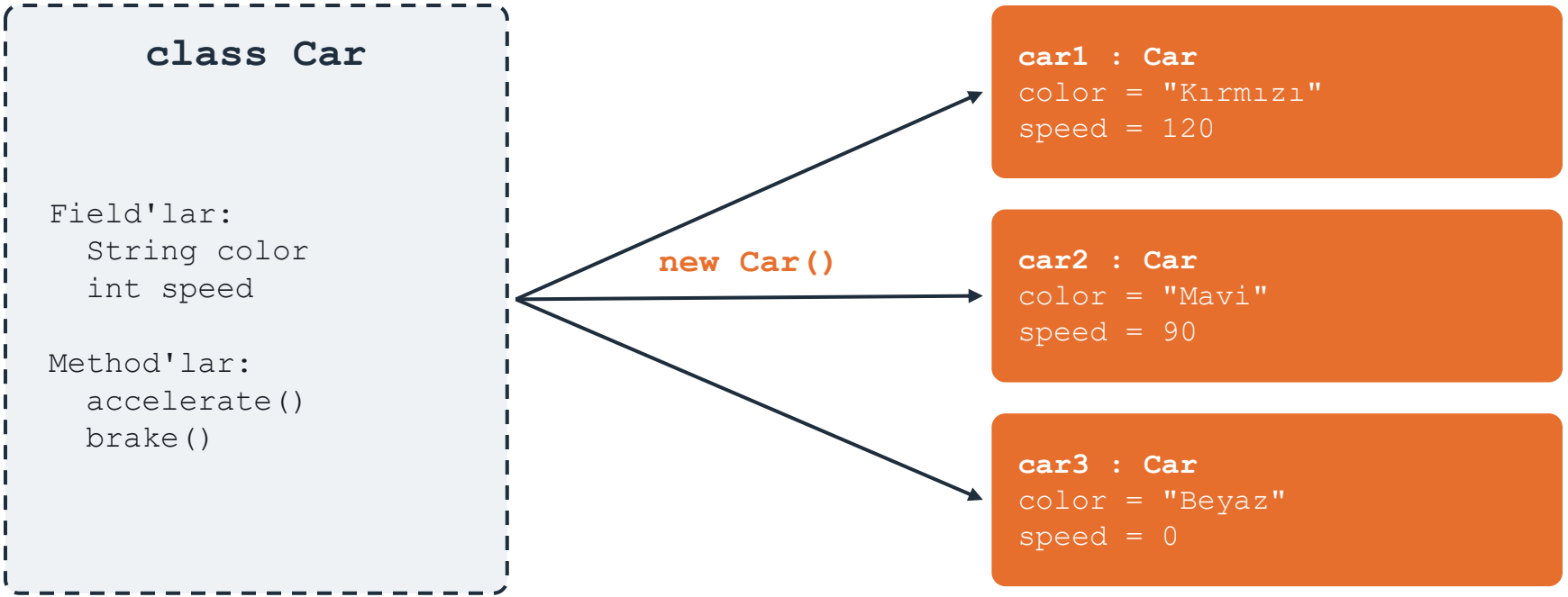
Bölüm 3



Class ve Object

Şablon ve ondan üretilen örnekler

Class = Blueprint, Object = Instance



Şablon: bellekte object yok

Object'ler: heap'te, kendi değerleriyle

Bir Class Neler İçerebilir?

Fields

Object'in durumunu tutan variable'lar (instance variable). `private`, `public` gibi modifier alabilir.

Methods

Object'in davranışları. Değer döndürebilir veya `void` olabilir.

Constructors

`new` ile object oluşturulurken çalışan, object'i ilk değerleriyle hazırlayan özel yapı.

Blocks

Instance block: her object oluşturulurken çalışır. **Static block**: class yüklenirken bir kez çalışır.

Nested class / interface

Class içinde tanımlanan class ve interface'ler; ilgili yapıları mantıksal olarak gruplar.

Field'ların Default Değerleri

```java
class Student {
    int id;  // field (instance variable)
    String name;
    double gpa;
    boolean active;
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();  // object oluştur
        System.out.println(s1.id);
        System.out.println(s1.name);
        System.out.println(s1.gpa);
        System.out.println(s1.active);
    }
}
```

Çıktı (Output)

```
0
null
0.0
false
```

- Değer verilmeyen field'lar **default** değer alır: sayılar `0`, `boolean` `false`, reference'lar `null`.
- Local variable'lar ise default değer **almaz** (önceki derste).
- Gerçek projelerde her class genellikle **ayrı bir .java dosyasında** tutulur; main ayrı bir class'tadır.

Object'e Değer Vermek: Reference ve Method ile

```java
class Student {
    int rollno;
    String name;
    void insertRecord(int r, String n) {    // method ile
        rollno = r; name = n;
    }
    void displayInformation() {
        System.out.println(rollno + " " + name);
    }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.rollno = 101;                    // reference ile
        s1.name = "Ali";
        Student s2 = new Student();
        s2.insertRecord(222, "Ayşe");       // method ile
        s1.displayInformation();
        s2.displayInformation();
    }
}
```

Çıktı (Output)

```
101 Ali
222 Ayşe
```

- Object initialization'ın 3 yolu: **reference variable**, **method**, **constructor** (bu dersin sonunda).
- Field'lara doğrudan atama encapsulation'ı bozar; method/constructor tercih edilir.

Reference Variable: Object'in Adresi

```java
class Point {
    int x, y;
}

public class Main {
    public static void main(String[] args) {
        Point p1 = new Point();
        p1.x = 5;
        Point p2 = p1;  // YENİ object yok, aynı adres!
        p2.x = 99;
        System.out.println(p1.x);

        Point p3 = new Point();
        p3.x = 99;
        System.out.println(p1 == p2);  // aynı object mi?
        System.out.println(p1 == p3);
    }
}
```

Çıktı (Output)

```
99
true
false
```

- **`p1`**, **`p2`**, **`p3`** object'in kendisi değil, object'e işaret eden **reference**'lardır.
- **`p2 = p1`** atamasıyla iki reference **aynı** object'i gösterir (aliasing).
- **`==`** reference'ları karşılaştırır: içerik aynı olsa da farklı object'ler için **`false`**.

Gerçek Dünya Örneği: Banka Hesabı

Account.java

```
class Account {
    int accNo;
    String name;
    double amount;
    void insert(int a, String n, double amt) {
        accNo = a; name = n; amount = amt;
    }
    void deposit(double amt) {
        amount += amt;
        System.out.println(amt + " yatırıldı");
    }
    void withdraw(double amt) {
        if (amount < amt) System.out.println("Yetersiz bakiye");
        else {
            amount -= amt;
            System.out.println(amt + " çekildi");
        }
    }
    void checkBalance() {
        System.out.println("Bakiye: " + amount);
    }
}
```

main() çıktısı

```
Bakiye: 1000.0
40000.0 yatırıldı
15000.0 çekildi
Yetersiz bakiye
Bakiye: 26000.0
```

- main içinde: `a1.insert(832345, "Ankit", 1000);` ardından `checkBalance()`, `deposit(40000)`, `withdraw(15000)`, `withdraw(50000)`, `checkBalance()`.
- Her method object'in **kendi** `amount` değerini değiştirir.

Anonymous Object ve Tek Satırda Birden Fazla Object

```java
class Calculation {
    void fact(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++) {
            fact *= i;
        }
        System.out.println("factorial is " + fact);
    }
}

public class Main {
    public static void main(String[] args) {
        Calculation c = new Calculation();   // referanslı
        c.fact(4);
        new Calculation().fact(5);           // anonymous object

        Calculation c1 = new Calculation(), c2 = new Calculation();
        c1.fact(3);
    }
}
```

Çıktı (Output)

```
factorial is 24
factorial is 120
factorial is 6
```

- **Anonymous object**: reference'a atanmayan, yalnızca **bir kez** kullanılan object.
- Kullanıldıktan sonra erişilemez hâle gelir; garbage collector toplar.
- Primitive'lerdeki gibi tek satırda virgülle birden fazla object oluşturulabilir.

Java'da Object Oluşturmanın Yolları



new keyword

- En yaygın yol: `new Student()`
- Bellek ayırır ve constructor'ı çağırır



Reflection

- Class çalışma zamanında bulunur, constructor'ı çağrılır
- Eski `newInstance()` deprecated



clone()

- Var olan object'in kopyası
- Varsayılan olarak **shallow copy**
- Class `Cloneable` olmalı



Deserialization

- Dosya/ağdan gelen byte'lardan object
- `ObjectInputStream` ile okunur



Factory method

- Object döndüren static method
- Örnek: `LocalDate.now()`

Class ile Object Arasındaki Farklar

Özellik	Class	Object
Tanım	Object'lerin şablonu (blueprint)	Class'ın bir instance'ı
Tür	Mantıksal varlık	Çalışma zamanında var olan varlık
Bellek	Class bilgisi JVM'e yüklenince bir kez (Method Area)	Her object için heap'te ayrı yer
Oluşturma	class keyword ile bir kez tanımlanır	new ile istenildiği kadar oluşturulur
İçerik	Field ve method tanımları	Field'ların gerçek değerleri
Ömür	Class yüklü kaldığı sürece	Reference'ı olduğu sürece (sonra GC)
Inheritance	extends ile genişletilebilir	Object'ten türetilemez

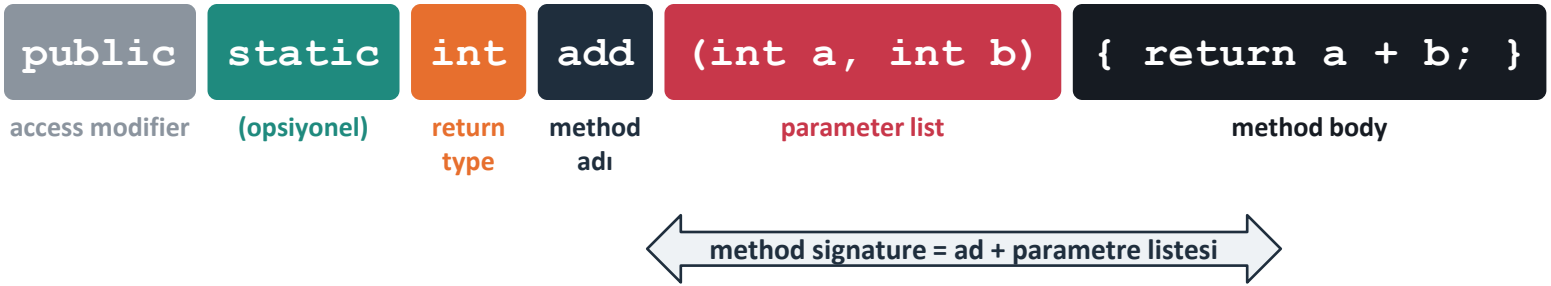
Bölüm 4



Methods

Kodu isimlendirilmiş, tekrar kullanılabilir parçalara bölmek

Method Declaration: Anatomi



- **Method**: belirli bir işi yapan, isimlendirilmiş kod bloğu; yalnızca **çağrıldığında** çalışır.
- **Return type**: döndürülen değerin tipi; değer döndürmüyorsa `void`.
- **Parameter list**: virgülle ayrılmış tip–isim çiftleri; parametre yoksa `()`.
- İsim fiil olmalı ve camelCase yazılmalı: `calculateArea()`, `isEmpty()`.
- Java'da her method bir class'ın içinde olmak **zorundadır** (C'deki serbest fonksiyonlar yoktur).

Method Kullanmanın Faydaları

Reusability

Bir kez yaz, birçok yerden çağır; aynı kodu tekrar tekrar yazma.

Modularity

Karmaşık problemi küçük, yönetilebilir parçalara böl.

Maintainability

Değişiklik ve hata ayıklama tek bir yerde yapılır.

Abstraction

Çağıran, method'un içini bilmeden kullanır: `Math.sqrt(16)`.

Access Modifier'lar: Kim Erişebilir?

Modifier	Aynı class	Aynı package	Farklı package'taki subclass	Her yer
`public`	✓	✓	✓	✓
`protected`	✓	✓	✓	✗
`default (yazılmazsa)`	✓	✓	✗	✗
`private`	✓	✗	✗	✗

Genel kural: field'lar private, dışarıya sunulan method'lar public olsun. En kısıtlayıcı uygun erişimi seçmek encapsulation'ın temelidir.

Predefined (Built-in) Methods

```java
System.out.println(Math.pow(2, 5));      // 2^5
System.out.println(Math.sqrt(144));
System.out.println(Math.max(17, 42));
System.out.println(Math.abs(-7.5));
System.out.println(Math.round(4.6));

String s = "Java";
System.out.println(s.length());
System.out.println(s.charAt(0));
System.out.println(s.toUpperCase());
System.out.println(s.equals("java"));
System.out.println(Integer.parseInt("25") + 5);
```

Çıktı (Output)

```
32.0
12.0
42
7.5
5
4
J
JAVA
false
30
```

- Java class library'sinde hazır gelen method'lar: **standard library / built-in** method.
- **`Math`** method'ları **static**'tir: class adıyla çağrılır.
- **`String`** method'ları ise object üzerinden çağrılır: **`s.length()`**.

User-Defined Method: static ve Instance

```java
public class Main {
    static int square(int x) {  // static method
        return x * x;
    }

    void greet(String name) {  // instance method
        System.out.println("Merhaba " + name);
    }

    public static void main(String[] args) {
        System.out.println(square(6));  // aynı class: doğrudan
        System.out.println(Main.square(3));  // class adıyla
        Main obj = new Main();
        obj.greet("Ayşe");  // object gerekir
        // greet("Ali");  -> HATA: static context'ten çağrılamaz
    }
}
```

Çıktı (Output)

```
36
9
Merhaba Ayşe
```

- **static** method'u çağırmak için object **gerekmez**: **`ClassName.method()`**.
- **Instance** method object üzerinden çağrılır: **`obj.method()`**.
- static bir method'dan instance method'u object olmadan çağırmak → **compile error**.

Parametreler ve return

```java
public class Main {
    static boolean isEven(int n) {
        return n % 2 == 0;              // boolean döndürür
    }
    static double average(int[] nums) {
        int sum = 0;
        for (int v : nums) sum += v;
        return (double) sum / nums.length;
    }
    static void printLine(int len) {    // void: değer yok
        for (int i = 0; i < len; i++) System.out.print("-");
        System.out.println();
    }
    public static void main(String[] args) {
        System.out.println(isEven(10));
        System.out.println(average(new int[]{3, 4, 8}));
        printLine(12);
    }
}
```

Çıktı (Output)

```
true
5.0
------------
```

- **`return`** değeri döndürür **ve** method'u hemen sonlandırır.
- Return type **`void`** olmayan method'un her yolu bir değer döndürmelidir.
- Çağıranın verdiği değerler **argument**, method tanımındakiler **parameter**'dır.

Java Her Zaman Pass-by-Value'dur

```java
public class Main {
    static void change(int x) {
        x = 100;  // kopya değişir
    }
    static void changeArray(int[] a) {
        a[0] = 100;  // aynı array değişir
    }
    public static void main(String[] args) {
        int n = 5;
        change(n);
        System.out.println(n);

        int[] arr = {5, 6};
        changeArray(arr);
        System.out.println(arr[0]);
    }
}
```

Çıktı (Output)

```
5
100
```

- Method'a her zaman değerin **kopyası** gider.
- Primitive'de kopya değer olduğu için orijinal değişmez.
- Reference'da kopyalanan şey **adrestir**: method aynı object'i/array'i değiştirebilir.

Bölüm 5



Constructors

Object'i doğduğu anda hazırlamak

Constructor Nedir? Kurallar



Tanım

- `new` ile object oluşturulurken **otomatik** çalışan özel blok
- Görevi: object'in field'larına ilk değerleri vermek
- Hiç yazılmazsa compiler bir **default constructor** ekler



Kurallar

- Adı class adıyla **birebir** aynı olmalı
- **Return type yoktur** (`void` bile yazılmaz)
- `abstract`, `static`, `final`, `synchronized` olamaz
- Access modifier alabilir: `public`, `protected`, default, `private`



Türler

- **Default**: compiler'ın eklediği
- **No-arg**: programcının yazdığı parametresiz
- **Parameterized**: parametre alan
- **Copy**: aynı class'tan bir object alıp kopyalayan

Dikkat: Return Type Yazılırsa Constructor Değildir

✗ void yazıldı: bu bir method

```java
class Test {
    void Test() {
        System.out.println("Ben bir method'um");
    }
}
public class Main {
    public static void main(String[] args) {
        Test t = new Test();  // çıktı yok
        t.Test();  // açıkça çağrılırsa
    }
}
```

Çıktı (Output)

```
Ben bir method'um
```

- Derlenir ama constructor **değildir**.

✓ Return type yok: constructor

```java
class Test {
    Test() {
        System.out.println("Constructor çalıştı");
    }
}
public class Main {
    public static void main(String[] args) {
        Test t = new Test();
    }
}
```

Çıktı (Output)

```
Constructor çalıştı
```

- **new** ile otomatik çağrılır; ayrıca çağırmaya gerek yoktur.

Default ve No-Arg Constructor

```java
class Bike {
    Bike() {  // no-arg constructor
        System.out.println("Bike is created");
    }
}

class Student {  // constructor yazılmadı:
    int id;  // compiler default
    String name;  // constructor ekler
}

public class Main {
    public static void main(String[] args) {
        Bike b = new Bike();
        Student s = new Student();
        System.out.println(s.id + " " + s.name);
    }
}
```

Çıktı (Output)

```
Bike is created
0 null
```

- **Default constructor**: class'ta hiç constructor yoksa compiler'ın eklediği boş, parametresiz constructor.
- `0` ve `null` değerleri constructor'dan değil, field'ların **default initialization**'ından gelir.
- `new` her zaman argümanlarla **eşleşen** constructor'ı çağırır.

Parameterized Constructor ve Constructor Overloading

```java
class Student {
    int id, age;
    String name;
    Student(int id, String name) {  // 2 parametre
        this.id = id; this.name = name;
    }
    Student(int id, String name, int age) {  // 3 parametre
        this(id, name);  // constructor chaining
        this.age = age;
    }
    void display() {
        System.out.println(id + " " + name + " " + age);
    }
}
public class Main {
    public static void main(String[] args) {
        new Student(111, "Karan").display();
        new Student(222, "Aryan", 25).display();
    }
}
```

Çıktı (Output)

```
111 Karan 0
222 Aryan 25
```

- Parameterized constructor, her object'e **farklı** değerlerle başlangıç verir.
- **Overloading**: parametre sayısı/tipi farklı birden fazla constructor.
- **`this(...)`** aynı class'taki başka bir constructor'ı çağırır (**constructor chaining**).

Sık Hata: Default Constructor Kaybolur

✗ Parametreli constructor var, no-arg yok

```
class Point {
    int x, y;
    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
Point p = new Point();        // HATA!
```

Derleme Hatası

```
error: constructor Point in class Point
cannot be applied to given types
```

- Constructor yazılınca default **eklenmez**.

✓ Gerekirse no-arg constructor'ı da yaz

```
class Point {
    int x, y;
    Point() { this(0, 0); }
    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
Point p = new Point();
System.out.println(p.x + "," + p.y);
```

Çıktı (Output)

```
0,0
```

- no-arg constructor, varsayılan değerlerle parametreli olanı çağırıyor.

Copy Constructor: Object'i Kopyalamak

```java
class Student {
    int id;
    String name;
    Student(int id, String name) {
        this.id = id; this.name = name;
    }
    Student(Student other) {          // copy constructor
        this.id = other.id;
        this.name = other.name;
    }
    void display() { System.out.println(id + " " + name); }
}
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(111, "Karan");
        Student s2 = new Student(s1);   // yeni, bağımsız object
        s2.name = "Aryan";
        s1.display();
        s2.display();
    }
}
```

Çıktı (Output)

```
111 Karan
111 Aryan
```

- Java'da C++'taki gibi **otomatik** copy constructor yoktur; gerekirse programcı yazar.
- `s2 = s1` kopya **değildir** (aynı object'i gösterir); copy constructor ise yeni object üretir.
- Alternatifler: `clone()` veya field'ları tek tek atamak.

Constructor ile Method Arasındaki Farklar

Constructor	Method
Object'in durumunu başlatır (initialize)	Object'in davranışını tanımlar
Return type'ı yoktur (void bile değil)	Mutlaka return type'ı vardır (void dahil)
new ile otomatik çağrılır	Açıkça çağrılır: obj.method()
Hiç yoksa compiler default constructor ekler	Compiler hiçbir method eklemez
Adı class adıyla aynı olmak zorunda	Adı herhangi bir geçerli isim olabilir
Inherit edilmez	Inherit edilir ve override edilebilir

Alıştırma: Çıktı Nedir?

1

Soru 1

```
class Box { int v; }
Box a = new Box();
Box b = a;
b.v = 7;
System.out.println(a.v);
```

2

Soru 2

```
class Box { int v; }
Box a = new Box();
Box b = new Box();
System.out.println(a == b);
```

3

Soru 3

```
class T {
    T() { System.out.print("C "); }
    void T() { System.out.print("M "); }
}
new T();
```

4

Soru 4

```
class Num {
    static void inc(int x) { x++; }
}
int n = 1;
Num.inc(n);
System.out.println(n);
```

Özet

OOP
Object (state + behavior + identity), class (blueprint); inheritance, polymorphism, abstraction, encapsulation.

Class ve Object
new heap'te object oluşturur; değişkenler object'in reference'ını tutar; == reference karşılaştırır.

Methods
Signature = ad + parametreler; static method object'siz çağrılır; Java pass-by-value'dur.

Constructors
Adı class ile aynı, return type yok; herhangi bir constructor yazılınca default constructor eklenmez.

Kaynak: tpointtech.com Java eğitim sayfaları (içerik genişletilmiş, örnekler eklenmiş ve hatalar düzeltilmiştir).